PROJECT DAWET

GitHub & Dokumentasi Proyek

Minggu 7 – Pra-Assignment 1

Learning Focus
Memahami Git dan GitHub, mengelola perubahan kode, mengunggah proyek ke repository, serta menyusun dokumentasi proyek menggunakan README.md.

Asynchronous Learning Program

Learn. Build. Understand.

Daftar Isi

1 Pendahuluan **2**
1.1 Tujuan Pembelajaran . . . 2

2 Memahami Git dan GitHub **2**
2.1 Apa Itu Git? . . . 2
2.2 Apa Itu GitHub? . . . 3
2.3 Perbedaan Git dan GitHub . . . 3

3 Konsep Dasar Git **4**
3.1 Repository . . . 4
3.2 Working Directory . . . 4
3.3 Staging Area . . . 4
3.4 Commit . . . 5
3.5 Remote . . . 5
3.6 Branch . . . 5

4 Mempersiapkan Git **5**
4.1 Menginstal Git . . . 5
4.2 Konfigurasi Identitas Git . . . 6
4.3 Menyiapkan Akun GitHub . . . 6

5 Membuat Repository di GitHub **7**

6 Menghubungkan Proyek Lokal ke GitHub **7**
6.1 Membuka Terminal pada Folder Proyek . . . 8
6.2 Menginisialisasi Repository Lokal . . . 8
6.3 Memeriksa Status Repository . . . 8
6.4 Memasukkan File ke Staging Area . . . 9
6.5 Membuat Commit Pertama . . . 9
6.6 Mengatur Nama Branch Utama . . . 9
6.7 Menghubungkan Repository Lokal dengan GitHub . . . 9
6.8 Mengunggah Proyek ke GitHub . . . 10

7 Alur Kerja Git Sehari-hari **10**
7.1 Menulis Pesan Commit yang Baik . . . 11

8 Mengenal README.md **11**
8.1 Apa Itu README? . . . 11
8.2 Mengapa README Penting? . . . 12
8.3 Contoh Struktur README . . . 12

8.4 Membuat README.md 13

9 Memahami dan Mengatasi Kendala Umum 14
9.1 Git Tidak Dikenali oleh Terminal 14
9.2 Author Identity Unknown 14
9.3 Remote Origin Already Exists 14
9.4 Push Ditolak karena Riwayat Berbeda 15
9.5 File Tidak Muncul di GitHub 15

10 Praktik Mandiri 16
10.1 Tugas Praktik 16
10.2 Ketentuan Praktik 16
10.3 Checklist Penyelesaian 17

11 Hubungan Materi dengan Assignment 1 17
11.1 Kriteria Pemeriksaan 17

12 Penutup 18

1. Pendahuluan

Ketika mengembangkan sebuah website, kode program biasanya tidak hanya terdiri dari satu file. Sebuah proyek dapat memiliki banyak file HTML, CSS, JavaScript, gambar, dan dokumen pendukung lainnya.

Seiring bertambahnya fitur, perubahan kode juga semakin sering dilakukan. Jika seluruh perubahan hanya disimpan secara lokal di komputer, pengembang dapat mengalami kesulitan ketika ingin melacak perubahan, mengembalikan versi sebelumnya, atau membagikan proyek kepada orang lain.

Dalam pengembangan perangkat lunak, terdapat dua teknologi yang sangat umum digunakan untuk membantu kebutuhan tersebut, yaitu Git dan GitHub.

Git digunakan untuk mencatat serta mengelola riwayat perubahan pada proyek. Sementara itu, GitHub merupakan platform berbasis web yang menyediakan layanan hosting repository Git dan berbagai fitur kolaborasi.

Pada materi ini, kita akan mempelajari cara menggunakan Git dan GitHub untuk mengelola proyek website sederhana yang telah dibuat pada materi sebelumnya.

1.1 Tujuan Pembelajaran

Setelah menyelesaikan materi ini, peserta diharapkan mampu:

1. Menjelaskan perbedaan Git dan GitHub.
2. Memahami konsep repository, commit, staging, dan remote repository.
3. Menginstal serta melakukan konfigurasi dasar Git.
4. Membuat repository di GitHub.
5. Menghubungkan proyek lokal dengan repository GitHub.
6. Mengunggah perubahan proyek menggunakan Git.
7. Menyusun dokumentasi proyek menggunakan README.md.

2. Memahami Git dan GitHub

2.1 Apa Itu Git?

Git adalah sistem version control terdistribusi (distributed version control system) yang digunakan untuk mencatat dan mengelola perubahan pada file proyek.

Dengan Git, pengembang dapat:

- Melihat riwayat perubahan kode.
- Mengetahui perubahan yang telah dilakukan.
- Menyimpan kondisi proyek pada titik tertentu.
- Membandingkan perubahan antarversi.
- Bekerja menggunakan branch tanpa harus mengubah versi utama secara langsung.

Sebagai contoh, ketika kita membuat website, kita mungkin melakukan perubahan berkali-kali. Git memungkinkan kita menyimpan riwayat perubahan tersebut sehingga perkembangan proyek dapat ditelusuri.

2.2 Apa Itu GitHub?

GitHub adalah platform berbasis web yang menyediakan layanan hosting repository Git.

GitHub memungkinkan pengembang untuk menyimpan repository secara daring, membagikan kode, berkolaborasi dengan pengembang lain, serta mengelola pekerjaan melalui fitur seperti Issues dan Pull Requests.

GitHub bukan pengganti Git. Git tetap merupakan sistem version control yang digunakan untuk mengelola perubahan, sedangkan GitHub menyediakan tempat daring untuk menyimpan dan mengelola repository Git.

2.3 Perbedaan Git dan GitHub

Aspek	**Git**	**GitHub**
Bentuk	Software version control	Platform hosting repository
Penggunaan	Mengelola riwayat perubahan	Menyimpan dan membagikan repository
Koneksi internet	Dapat digunakan secara lokal	Membutuhkan internet untuk layanan daring
Contoh aktivitas	Commit, branch, merge	Membuat repository, Pull Request

Analogi Sederhana

Bayangkan Git sebagai sistem pencatatan versi dokumen di komputer. Git mencatat setiap perubahan yang kita simpan.
Sementara itu, GitHub dapat diibaratkan sebagai tempat penyimpanan daring yang memungkinkan dokumen tersebut diakses dan dibagikan kepada orang lain.

3. Konsep Dasar Git

Sebelum menggunakan perintah Git, kita perlu memahami beberapa istilah yang akan sering digunakan.

3.1 Repository

Repository atau sering disebut repo adalah tempat penyimpanan proyek yang dikelola oleh Git.

Repository tidak hanya menyimpan file proyek, tetapi juga menyimpan informasi mengenai riwayat perubahan yang telah dicatat oleh Git.

Terdapat dua jenis repository yang akan kita gunakan:

- **Local Repository**: repository yang berada di komputer kita.
- **Remote Repository**: repository yang berada di layanan daring seperti GitHub.

3.2 Working Directory

Working directory adalah direktori proyek tempat kita bekerja dan melakukan perubahan terhadap file.

Contohnya, ketika kita mengedit `index.html`, perubahan tersebut pertama kali terjadi pada working directory.

3.3 Staging Area

Staging area adalah area sementara untuk memilih perubahan yang akan dimasukkan ke dalam commit berikutnya.

Kita tidak harus memasukkan seluruh perubahan sekaligus. Git memungkinkan kita memilih file tertentu untuk dipersiapkan sebelum commit.

3.4 Commit

Commit adalah catatan perubahan yang disimpan dalam riwayat repository.

Setiap commit memiliki identitas unik dan pesan yang menjelaskan perubahan yang dilakukan.

Contoh pesan commit:

```
Menambahkan halaman profil UMKM
```

3.5 Remote

Remote adalah referensi menuju repository lain yang terhubung dengan repository lokal.

Dalam materi ini, remote akan mengarah ke repository GitHub yang kita buat.

3.6 Branch

Branch merupakan jalur pengembangan yang memungkinkan kita mengerjakan perubahan secara terpisah dari jalur utama proyek.

Untuk tahap awal, kita akan menggunakan branch bernama `main` sebagai branch utama.

4. Mempersiapkan Git

4.1 Menginstal Git

Git dapat diunduh melalui situs resmi:

https://git-scm.com/downloads

Pilih sistem operasi yang digunakan, kemudian ikuti langkah instalasi sesuai petunjuk yang tersedia.

Pada Windows, pengaturan instalasi bawaan umumnya sudah cukup untuk kebutuhan pembelajaran ini.

Setelah instalasi selesai, buka terminal seperti Command Prompt, PowerShell, atau Git Bash.

Periksa apakah Git berhasil terpasang dengan perintah:

```
git --version
```

Jika instalasi berhasil, terminal akan menampilkan versi Git yang terpasang.

4.2 Konfigurasi Identitas Git

Git menggunakan identitas pengguna untuk mencatat siapa yang membuat sebuah commit.

Atur nama pengguna:

```
git config --global user.name "Nama Kamu"
```

Atur alamat email:

```
git config --global user.email "email@example.com"
```

Gantilah nama dan alamat email tersebut dengan identitas yang ingin digunakan pada riwayat commit.

Periksa konfigurasi dengan:

```
git config --global --list
```

Perlu diperhatikan bahwa konfigurasi identitas Git tidak otomatis melakukan autentikasi ke akun GitHub. Keduanya merupakan pengaturan yang berbeda.

4.3 Menyiapkan Akun GitHub

Jika belum memiliki akun GitHub, lakukan pendaftaran melalui:

https://github.com/

Gunakan alamat email aktif dan ikuti proses verifikasi yang diminta.

Ketika melakukan push melalui HTTPS, GitHub dapat meminta autentikasi melalui browser atau mekanisme credential manager yang tersedia.

GitHub tidak menerima kata sandi akun biasa sebagai metode autentikasi Git melalui HTTPS. Ikuti metode autentikasi resmi yang ditawarkan oleh Git atau GitHub.

5. Membuat Repository di GitHub

Setelah memiliki akun GitHub, kita dapat membuat repository untuk menyimpan proyek.

Ikuti langkah berikut:

1. Buka `https://github.com/`.
2. Login menggunakan akun GitHub.
3. Klik tombol *New* untuk membuat repository.
4. Isi nama repository, misalnya `dawet-assignment-1-nama`.
5. Tambahkan deskripsi singkat mengenai proyek.
6. Pilih visibilitas repository sesuai kebutuhan pengumpulan tugas.
7. Untuk latihan ini, gunakan repository kosong tanpa menambahkan README, .gitignore, atau lisensi terlebih dahulu.
8. Klik *Create repository*.

Perhatian: Visibilitas Repository

Repository dengan status Public dapat dilihat oleh siapa saja.
Jangan pernah mengunggah kata sandi, API key, token, informasi pribadi, atau file rahasia ke repository Public maupun repository yang dibagikan kepada orang lain.
Pastikan seluruh file yang diunggah memang aman untuk dibagikan.

Setelah repository berhasil dibuat, GitHub akan menampilkan halaman repository beserta alamat URL yang dapat digunakan untuk menghubungkan repository lokal.

6. Menghubungkan Proyek Lokal ke GitHub

Pada bagian ini, kita akan mengunggah proyek website yang telah dibuat pada materi HTML dan CSS sebelumnya.

Pastikan folder proyek sudah tersedia di komputer.

Contoh struktur folder:

```
website-umkm/
|
|-- index.html
|-- tentang.html
```

```
|-- produk.html
|-- kontak.html
|
|-- css/
|   |-- style.css
|
|-- images/
    |-- logo.png
```

Struktur di atas hanya contoh. Gunakan struktur proyek yang memang dimiliki masing-masing peserta.

6.1 Membuka Terminal pada Folder Proyek

Buka terminal, kemudian arahkan ke folder proyek.

Contoh:

```
cd Documents/website-umkm
```

Pastikan terminal berada di direktori yang benar sebelum menjalankan perintah Git.

6.2 Menginisialisasi Repository Lokal

Jalankan:

```
git init
```

Perintah tersebut menginisialisasi repository Git pada folder proyek.

Jika folder tersebut sudah menjadi repository Git, tidak perlu menginisialisasinya kembali.

6.3 Memeriksa Status Repository

Jalankan:

```
git status
```

Perintah ini menampilkan kondisi repository, termasuk file yang belum dilacak dan perubahan yang belum dimasukkan ke staging area.

6.4 Memasukkan File ke Staging Area

Jalankan:

```
git add .
```

Perintah tersebut memasukkan perubahan pada direktori kerja saat ini beserta subdirektorinya ke staging area.

Sebelum menjalankan perintah ini, pastikan tidak ada file rahasia atau file yang tidak seharusnya diunggah.

Periksa kembali hasilnya:

```
git status
```

6.5 Membuat Commit Pertama

Jalankan:

```
git commit -m "Initial commit"
```

Perintah ini menyimpan perubahan yang sudah berada di staging area sebagai commit.

Pesan `Initial commit` biasanya digunakan untuk menandai penyimpanan awal proyek.

6.6 Mengatur Nama Branch Utama

Jalankan:

```
git branch -M main
```

Perintah ini mengganti nama branch aktif menjadi `main`.

Perintah tersebut tidak menghapus file atau mengubah isi proyek.

6.7 Menghubungkan Repository Lokal dengan GitHub

Salin URL repository dari halaman GitHub.

Contoh URL:

```
https://github.com/username/nama-repository.git
```

Kemudian jalankan:

```
git remote add origin https://github.com/username/nama-repository.git
```

Gantilah URL contoh tersebut dengan URL repository milik masing-masing peserta.

Periksa remote yang telah ditambahkan:

```
git remote -v
```

Jika berhasil, terminal akan menampilkan alamat repository remote untuk operasi fetch dan push.

6.8 Mengunggah Proyek ke GitHub

Jalankan:

```
git push -u origin main
```

Perintah tersebut mengunggah commit pada branch `main` ke remote bernama `origin`.

Opsi `-u` mengatur upstream branch sehingga pada penggunaan berikutnya kita dapat menjalankan `git push` tanpa harus menuliskan remote dan branch secara lengkap.

Jika diminta autentikasi, ikuti instruksi yang ditampilkan oleh Git atau browser.

Setelah proses berhasil, buka kembali halaman repository GitHub dan periksa apakah file proyek sudah muncul.

7. Alur Kerja Git Sehari-hari

Setelah repository berhasil terhubung, kita tidak perlu mengulang seluruh proses dari awal setiap kali melakukan perubahan.

Alur kerja dasar yang akan sering digunakan adalah:

1. Mengubah atau menambahkan file proyek.
2. Memeriksa perubahan menggunakan `git status`.
3. Memasukkan perubahan menggunakan `git add`.
4. Menyimpan perubahan menggunakan `git commit`.
5. Mengunggah perubahan menggunakan `git push`.

Contoh:

```
git status
```

```
git add .
git commit -m "Memperbaiki tampilan halaman utama"
git push
```

Dengan alur tersebut, perubahan terbaru dapat disimpan dalam riwayat Git dan dikirim ke GitHub.

7.1 Menulis Pesan Commit yang Baik

Pesan commit sebaiknya menjelaskan perubahan yang dilakukan secara singkat dan jelas.

Contoh pesan yang baik:

```
Menambahkan halaman produk
Memperbaiki navigasi website
Mengatur layout kartu produk
Memperbaiki tampilan mobile
```

Hindari pesan yang terlalu umum seperti:

```
Update
Perbaikan
Coba lagi
Final
```

Pesan commit yang jelas akan mempermudah kita memahami riwayat perkembangan proyek.

8. Mengenal README.md

8.1 Apa Itu README?

README adalah dokumen pengantar yang menjelaskan informasi penting mengenai sebuah proyek.

Pada repository GitHub, file `README.md` biasanya ditampilkan secara otomatis pada halaman utama repository.

Format `.md` merupakan Markdown, yaitu format penulisan teks sederhana yang mendukung judul, paragraf, daftar, tautan, gambar, dan beberapa elemen dokumentasi lainnya.

8.2 Mengapa README Penting?

README membantu orang lain memahami proyek tanpa harus membaca seluruh kode program terlebih dahulu.

Dokumen ini dapat menjelaskan:

- Nama dan tujuan proyek.
- Fitur yang tersedia.
- Teknologi yang digunakan.
- Cara menjalankan proyek.
- Struktur folder.
- Informasi pengembang.

README juga membantu pengembang mengingat cara kerja proyek yang sedang dikerjakan.

8.3 Contoh Struktur README

Berikut contoh struktur README sederhana:

```
# Website UMKM

Website sederhana untuk memperkenalkan
produk dan informasi UMKM.

## Deskripsi

Proyek ini dibuat sebagai bagian dari
Assignment 1 Project DAWET.

## Fitur

- Halaman utama
- Halaman profil UMKM
- Daftar produk
- Informasi kontak

## Teknologi

- HTML
- CSS
```

```
- JavaScript (jika digunakan)

## Struktur Proyek

website-umkm/
|-- index.html
|-- tentang.html
|-- produk.html
|-- kontak.html
|-- css/
|-- images/

## Cara Menjalankan

1. Clone repository.
2. Buka folder proyek.
3. Jalankan index.html melalui browser.

## Pengembang

Nama: Nama Peserta
Program: Project DAWET
```

Contoh di atas dapat disesuaikan dengan kondisi proyek masing-masing peserta. Jangan mencantumkan fitur atau teknologi yang sebenarnya tidak digunakan.

8.4 Membuat README.md

Buat file baru bernama:

```
README.md
```

Simpan file tersebut pada direktori utama proyek, sejajar dengan `index.html`.

Isi file menggunakan struktur README yang telah dipelajari, kemudian simpan.

Setelah itu, masukkan perubahan ke Git:

```
git add README.md
git commit -m "Menambahkan dokumentasi proyek"
git push
```

Buka kembali repository GitHub untuk memastikan README berhasil ditampilkan.

9. Memahami dan Mengatasi Kendala Umum

Dalam penggunaan Git, beberapa kendala mungkin muncul, terutama ketika pertama kali melakukan konfigurasi atau menghubungkan repository.

Berikut beberapa masalah yang sering ditemukan.

9.1 Git Tidak Dikenali oleh Terminal

Contoh pesan:

```
'git' is not recognized as an internal
or external command
```

Penyebab yang umum adalah Git belum terinstal atau lokasi instalasinya belum masuk ke PATH.

Solusi:

1. Pastikan Git sudah terinstal.
2. Tutup dan buka kembali terminal.
3. Periksa kembali dengan `git -version`.
4. Jika masih gagal, periksa konfigurasi PATH.

9.2 Author Identity Unknown

Contoh pesan:

```
Author identity unknown
```

Git belum memiliki konfigurasi nama dan email.

Solusi:

```
git config --global user.name "Nama Kamu"
git config --global user.email "email@example.com"
```

Setelah itu, ulangi proses commit.

9.3 Remote Origin Already Exists

Contoh pesan:

```
error: remote origin already exists.
```

Artinya, remote bernama `origin` sudah terdaftar pada repository lokal.

Periksa remote yang tersedia:

```
git remote -v
```

Jika URL yang ditampilkan sudah benar, tidak perlu menambahkan remote kembali.

Jika URL-nya salah, perbarui dengan:

```
git remote set-url origin URL_REPOSITORY
```

Gantilah `URL_REPOSITORY` dengan URL repository yang benar.

9.4 Push Ditolak karena Riwayat Berbeda

Contoh pesan:

```
! [rejected] main -> main (fetch first)
```

Kondisi ini dapat terjadi ketika repository GitHub sudah memiliki commit yang belum dimiliki oleh repository lokal.

Salah satu penyebabnya adalah membuat repository GitHub dengan README terlebih dahulu, kemudian membuat commit awal yang berbeda secara lokal.

Untuk pemula, cara paling sederhana adalah menghindari kondisi tersebut sejak awal dengan membuat repository GitHub kosong.

Jika masalah sudah terjadi, jangan langsung menggunakan `git push -force` atau menghapus folder `.git`.

Periksa kondisi repository terlebih dahulu. Jika tidak yakin, mintalah bantuan sebelum menjalankan perintah yang dapat mengubah riwayat repository.

9.5 File Tidak Muncul di GitHub

Periksa beberapa hal berikut:

- Apakah file berada di dalam folder proyek?
- Apakah file sudah dimasukkan ke staging area?
- Apakah perubahan sudah di-commit?

- Apakah commit sudah di-push?
- Apakah URL repository yang digunakan sudah benar?

Gunakan perintah berikut untuk memeriksa kondisi:

```
git status
git remote -v
git log --oneline
```

Perintah tersebut membantu mengetahui status file, alamat remote, dan riwayat commit.

10. Praktik Mandiri

Pada bagian ini, peserta akan menerapkan seluruh materi dengan mengunggah proyek website yang sudah dibuat pada pembelajaran sebelumnya.

10.1 Tugas Praktik

Lakukan langkah berikut:

1. Siapkan proyek HTML dan CSS yang telah dibuat.
2. Pastikan struktur folder proyek rapi.
3. Buat repository baru di GitHub.
4. Hubungkan proyek lokal dengan repository GitHub.
5. Unggah seluruh file proyek.
6. Buat file `README.md`.
7. Lakukan minimal dua commit dengan pesan yang menjelaskan perubahan secara jelas.
8. Pastikan seluruh perubahan sudah muncul pada repository GitHub.

10.2 Ketentuan Praktik

- Repository harus dapat diakses oleh ketua untuk proses pemeriksaan.
- Gunakan nama repository yang jelas.
- README harus sesuai dengan kondisi proyek.
- Jangan mengunggah file rahasia atau tidak relevan.

- Pastikan file HTML, CSS, dan aset gambar dapat ditemukan melalui repository.

10.3 Checklist Penyelesaian

Gunakan checklist berikut untuk memastikan praktik telah selesai.

Git berhasil terinstal.

Identitas Git sudah dikonfigurasi.

Repository GitHub berhasil dibuat.

Proyek lokal berhasil terhubung ke GitHub.

File proyek berhasil diunggah.

README.md berhasil dibuat.

Minimal dua commit berhasil dibuat.

Repository dapat dibuka dan diperiksa.

11. Hubungan Materi dengan Assignment 1

Materi ini merupakan bagian dari persiapan Assignment 1: Web Foundation.

Selain menghasilkan website sederhana menggunakan HTML dan CSS, peserta juga perlu memahami bagaimana cara mengelola serta mendokumentasikan proyeknya.

GitHub digunakan sebagai media pengumpulan repository, sedangkan README membantu ketua memahami isi dan tujuan proyek yang dikumpulkan.

Dalam proses evaluasi, peserta tidak hanya dinilai berdasarkan hasil tampilan website, tetapi juga berdasarkan kerapian repository dan kemampuan menjelaskan pekerjaan yang telah dilakukan.

11.1 Kriteria Pemeriksaan

Berikut aspek yang akan diperiksa:

Aspek	Indikator
Repository	Repository tersedia dan dapat diakses.
Struktur	File proyek tersusun dengan jelas.
Commit	Riwayat perubahan dapat dilihat dan memiliki pesan yang relevan.
README	Dokumentasi menjelaskan proyek secara benar.
Pemahaman	Peserta mampu menjelaskan fungsi Git, GitHub, dan alur kerja yang digunakan.

12. Penutup

Git dan GitHub merupakan perangkat penting dalam pengembangan perangkat lunak modern.

Git membantu pengembang mengelola perubahan kode, sedangkan GitHub membantu menyimpan dan membagikan repository secara daring.

Melalui praktik pada materi ini, peserta diharapkan mulai terbiasa mengelola proyek secara lebih terstruktur, tidak hanya menyimpan file di komputer, tetapi juga mencatat perkembangan pekerjaan dan menyediakan dokumentasi yang dapat dipahami orang lain.

Ingat bahwa tujuan utama penggunaan Git bukan sekadar menghafal perintah, melainkan memahami bagaimana perubahan proyek dikelola dengan baik.

Learn. Build. Understand.

PROJECT DAWET

Referensi Resmi

Materi lanjutan dapat dipelajari melalui dokumentasi resmi berikut:

1. Git Documentation: `https://git-scm.com/doc`
2. Git Book: `https://git-scm.com/book/en/v2`
3. GitHub Documentation: `https://docs.github.com/`
4. GitHub – About Git: `https://docs.github.com/en/get-started/using-git/about-git`
5. GitHub – Creating a New Repository: `https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-new-repository`

6. GitHub – Hello World: https://docs.github.com/en/get-started/start-your-journey/hello-world